

Advanced Stream Operations in Java 8

👉 `flatMap()` | `groupingBy()` |
`partitioningBy()`

Take your functional programming to the next level.

SWIPE ➞

Why Advanced Stream Operations?

Basic Stream operations → `map()`, `filter()`, `collect()`

But REAL applications require :

- ✓ Transforming nested data
- ✓ Categorizing items
- ✓ Grouping results
- ✓ Aggregating large datasets

➡ That's where `flatMap()`, `groupingBy()`, `partitioningBy()` become powerful

flatMap() — Flatten + Map

? Problem :

What if you have a list inside another list?

Example :

```
List<List<Integer>> nums =  
Arrays.asList(  
    Arrays.asList(1, 2),  
    Arrays.asList(3, 4)  
);
```

! map() will give → Stream<List<Integer>>

But we need → Stream<Integer>

✓ flatMap() solves this :

```
List<Integer> result =  
nums.stream()  
    .flatMap(list -> list.stream())  
    .collect(Collectors.toList());
```

Output : [1, 2, 3, 4]

flatMap = flatten + transform → solves nested structure problems.